

Reinforcement Learning for Knowledge Graph Co-optimization: A Dual-Strategy Rule Generation Framework

Anonymous Authors

Abstract—Knowledge-graph quality control couples two decisions that are usually treated separately: which constraints to acquire and which detected graph defects to repair under a limited budget. We formulate this interaction as a finite-horizon Markov decision process and learn a Double-DQN meta-policy over four executable graph operations and four rule-management operations. A complementary dual-strategy LLM module expands the candidate rule space through deletion completion and augmentation expansion. In an executable controlled-corruption benchmark with actual graph mutations, ten independently trained policies achieve normalized final joint quality of 0.9812 ± 0.0013 . Relative to the strongest non-oracle fixed schedule, Double DQN improves final quality by 0.0065 and trajectory AUC by 0.0166 ($p = 0.000977$ for both paired one-sided Wilcoxon tests) while halving rule-generation calls from eight to four. Archived generation logs show that combining the two strategies yields 71.9% more canonicalized candidates than the stronger single strategy. On the designed RuleTest-94 suite, the two executable rule families cover disjoint defect groups and their union detects all 64 defective cases without flagging the 30 clean cases. Deterministic and API-backed extraction tests further characterize structural repair, while explicit scope controls distinguish controlled-suite coverage from open-world accuracy.

Index Terms—Knowledge graph quality, rule generation, reinforcement learning, Double DQN, graph repair, large language models.

I. Introduction

A. Research Background and Motivation

Knowledge Graph (KG) construction aims to extract structured knowledge from unstructured text. However, two critical bottlenecks persist:

- 1) **Static enhancement strategies**: Existing quality control methods follow fixed pipelines without adaptive decision-making—for example, always running entity completion before rule checking, regardless of current graph or rule quality. Systems cannot intelligently balance between immediate graph correction and long-term rule optimization based on current quality states.
- 2) **Limited rule coverage**: Quality assessment relies on expert-crafted rules, which is equivalent to manually sampling a few points in a huge "Rule Space". This leads to insufficient coverage, high maintenance costs, and poor domain transferability.

Recent advances in Large Language Models (LLMs) and Reinforcement Learning (RL) provide new opportunities.

LLMs can automatically explore rule spaces through contextual reasoning, while RL can learn adaptive policies for sequential decision-making under dynamic environments.

B. Contributions of This Paper

We make three contributions:

- 1) **An executable co-optimization MDP**. We define a 14-dimensional observable state, eight graph/rule actions, action feasibility masks, and a cost-aware joint reward. Every graph action mutates node or relation records and triggers metric recomputation. This turns the method into a reproducible sequential-control problem rather than a prompt-selection heuristic.
- 2) **A trained and fully archived Double-DQN policy**. The implementation includes an online network, target network, replay buffer, Double-DQN Bellman target, and epsilon-greedy exploration. Across ten paired seeds, the policy improves final normalized joint quality by 0.0065 and trajectory AUC by 0.0166 over the strongest fixed schedule, with paired one-sided Wilcoxon $p = 0.000977$ for both. It uses four rule-generation calls compared with eight for that schedule.
- 3) **Dual-strategy rule-space expansion with separated evidence**. Deletion completion and augmentation expansion contribute complementary stored candidates: their union contains 71.9% more canonicalized candidates than the stronger individual strategy in the full government log. We separately evaluate executable family coverage on RuleTest-94, structural constraint checking, deterministic repair, and API-backed extraction, avoiding the conflation of candidate count, designed-suite detection, and open-world accuracy.

The evaluation retains a model-informed one-step greedy policy as an upper-bound diagnostic. Double DQN approaches this upper bound without observing the transition function, while the limitations section states where controlled corruption and a designed rule suite restrict external validity.

C. Paper Organization

The rest of this paper is organized as follows: Section 2 reviews related work in RL for KG and rule generation. Section 3 elaborates on the RL optimization framework and dual-strategy rule generation. Section 4 presents experimental design and results. Section 5 concludes the paper.

Figure 1 provides a high-level overview of the proposed framework. The RL agent observes the joint quality state $[S(C_1), S(C_2), S(C_3), S(C_4), \text{Precision}, \text{Recall}, \text{Coverage}]$, selects either a graph-enhancement action or a rule-generation action, executes it, and receives a reward proportional to the quality improvement. This closed loop repeats until both graph quality and online rule quality reach target thresholds or the episode budget is exhausted.

II. Related Work

A. Reinforcement Learning for Knowledge Graphs

RL for Knowledge Graph Reasoning. Reinforcement Learning has emerged as a promising paradigm for KG reasoning and path finding. Das et al. [1] pioneered RL application to multi-hop reasoning, using policy gradient methods (REINFORCE) to learn path-finding strategies over KG structures. DeepPath [2] introduced MINERVA for walking over KG structures using policy networks. Lin et al. [3] proposed multi-hop reasoning with reward shaping to guide exploration. DIVA [4] uses variational inference for interpretable reasoning paths. These methods focus on link prediction and query answering but do not address quality enhancement.

RL for Knowledge Graph Construction. Recent work applies RL to KG construction tasks. Chen et al. [5] applied Deep Q-Networks to optimize trade-offs between graph coverage and precision during incremental construction. Zhu et al. [6] use RL for entity alignment across multilingual KGs. Adaptive KG construction [7] employs actor-critic methods to select extraction strategies. However, these approaches focus on construction pipelines rather than holistic quality enhancement involving rule generation and multi-dimensional optimization.

RL for Sequential Decision-Making in NLP. RL has been successfully applied to various NLP tasks requiring sequential decisions. Ranzato et al. [8] use REINFORCE for sequence-to-sequence learning. Paulus et al. [9] apply RL to abstractive summarization with policy gradient methods. Li et al. [10] use RL for dialogue generation. These works demonstrate RL’s effectiveness in learning long-term strategies, inspiring our application to KG quality optimization.

B. Large Language Models for Knowledge Engineering

LLM-Based Knowledge Extraction. Large Language Models have revolutionized KG construction through various paradigms. Zero-shot/few-shot prompting [11] extracts triples via natural language instructions without training. GPT-4 [12] and Mistral-7B [13] achieve competitive performance on WebNLG datasets. Fine-tuning methods like QLoRA [14] reduce memory usage while maintaining performance. Retrieval-augmented generation [15] combines LLMs with external knowledge to reduce hallucinations.

LLM-Based Knowledge Completion. LLMs are increasingly used for KG completion tasks. KG-BERT [16] fine-tunes BERT for triple classification. KEPLER [17] jointly optimizes language modeling and KG embedding. Recent work [18] explores unifying LLMs and KGs through bidirectional enhancement. However, these methods focus on fact completion rather than rule generation.

TABLE I
Comparison with Related Work

Method	RL	LLM	Auto Rules	Task Focus
Das et al. [1]	Yes	No	No	Reasoning
Chen et al. [5]	Yes	No	No	Construction
AMIE [25]	No	No	Yes	Rule mining
Lin et al. [28]	No	Yes	No	Repair
Our Method	Yes	Yes	Yes	Quality optimization

LLM-Based Reasoning and Planning. LLMs demonstrate strong reasoning capabilities. Chain-of-Thought prompting [11] elicits step-by-step reasoning. ReAct [19] combines reasoning and acting for complex tasks. Tree-of-Thoughts [20] explores multiple reasoning paths. These capabilities inspire our use of LLMs for rule discovery through contextual reasoning.

C. Rule-Based Quality Assessment and Generation

Manual Rule Curation. Traditional rule-based methods rely on expert knowledge to define entity types, relationship constraints, and logical axioms. DBpedia [21] and YAGO [22] use handcrafted rules for schema validation. SHACL [23] and ShEx [24] provide languages for expressing KG constraints. However, manual rule curation is labor-intensive, domain-specific, and limited in coverage.

Automatic Rule Mining. Several approaches attempt to automatically mine rules from KGs. AMIE [25] mines Horn rules from RDF KGs using association rule mining. RuDiK [26] discovers positive and negative rules through iterative refinement. Neural rule learning [27] uses differentiable logic for rule induction. However, these methods mine rules from existing KG structure rather than from source text, limiting their ability to discover quality assessment rules.

Violation Detection and Repair. Recent work focuses on detecting and repairing KG violations. Violation-inducing operations (VIOs) [28] systematically test repair systems by introducing controlled errors. Constraint-based repair [29] uses statistical methods to fix inconsistencies. However, these approaches still require manual rule design and lack systematic rule generation mechanisms.

D. Comparison with Our Approach

Table I compares our work with representative prior approaches:

Our work uniquely combines RL-based optimization with LLM-driven rule generation for end-to-end quality enhancement. Unlike prior RL methods that focus on reasoning or construction, we address holistic quality optimization. Unlike rule mining approaches that extract rules from KG structure, we generate quality assessment rules from source text through LLM reasoning. This integration enables collaborative evolution of rules and knowledge graphs.

III. Methodology

A. Framework Overview

The framework maintains a graph G_t and an executable rule set R_t . At decision step t , a quality assessor converts their

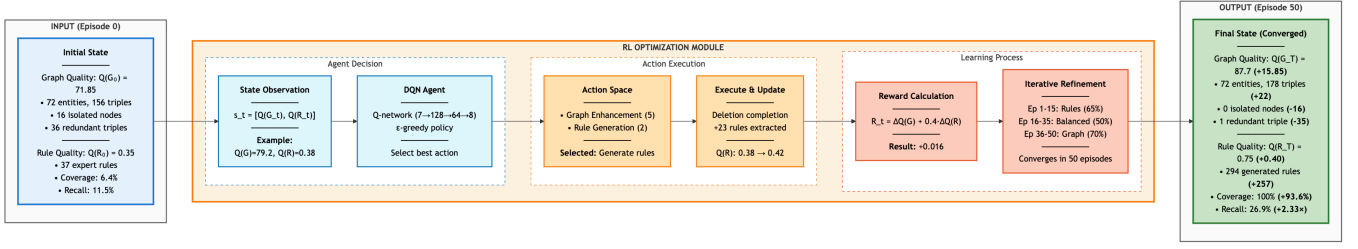


Fig. 1. Overview of the RL-Driven Co-Optimization Framework. The agent alternates between graph enhancement and rule generation actions, guided by joint quality signals $Q(G)$ and $Q_{\text{online}}(R)$. Section III describes each component in detail.

current status into an observable state s_t ; a value-based policy selects either a rule-acquisition operation or a graph-repair operation; the selected operation changes R_t or the graph object itself; and the assessor recomputes the state and reward. Rule acquisition and graph repair are therefore coupled: a repair action is executable only after its corresponding validator is active, while the value of acquiring a validator depends on defects that remain in the graph.

The main policy experiment uses an executable controlled-corruption environment built from TNEWS/CLUE records. This choice separates the sequential-control question from LLM sampling variance. Graph actions operate on actual node and relation records, and rule actions activate executable validation modules. The hidden corruption manifest is used only to apply a correct relation restoration after a validator identifies an invalid relation; it is never included in the policy state. Stored dual-strategy outputs are evaluated separately in Section IV.

B. Quality Assessment Model

1) *Knowledge Graph Quality Metrics*: For the executable environment, the four graph components are computed after every graph mutation:

$$\begin{aligned} S(C_1) &= 1 - I_r, & S(C_2) &= 1 - D_r, \\ S(C_3) &= 1 - V_r, & S(C_4) &= 1 - L_r, \end{aligned} \quad (1)$$

where I_r is the isolated-node rate, D_r is the exact-duplicate relation rate, V_r is the schema-invalid relation rate, and L_r is the dangling-edge rate. All four quantities are obtained by deterministic validators and lie in $[0, 1]$. We aggregate them as

$$Q(G_t) = 0.20S(C_1) + 0.20S(C_2) + 0.30S(C_3) + 0.30S(C_4). \quad (2)$$

These weights assign more mass to relation validity and referential integrity because those violations can change graph semantics or make a triple unusable.

2) *Rule Set Quality Metrics*: The online rule metrics are evaluated on a fixed validation inventory that is disjoint from the graph corruption manifest. For the union of predictions made by the active validators,

$$\text{Precision}(R) = \frac{TP(R)}{TP(R) + FP(R)}, \quad \text{Recall}(R) = \frac{TP(R)}{TP(R) + FN(R)} \quad (3)$$

Coverage is the fraction of the four benchmark defect families for which an executable validator is active:

$$\text{Coverage}(R) = \frac{|\{c : \exists r \in R \text{ that detects } c\}|}{4}. \quad (4)$$

The online score is

$$Q_{\text{online}}(R) = 0.35 \text{ Precision}(R) + 0.35 \text{ Recall}(R) + 0.30 \text{ Coverage}(R). \quad (5)$$

For the separate rule-generation study, we additionally report unique detection capability,

$$\text{Unique}(R) = \frac{|\{x : x \text{ is detected only by } R\}|}{|\{x : x \text{ is a true defect}\}|}, \quad (6)$$

and use it only as an offline diagnostic:

$$Q_{\text{eval}}(R) = 0.30 \text{ Precision} + 0.30 \text{ Recall} + 0.25 \text{ Coverage} + 0.15 \text{ Unique}. \quad (7)$$

C. Reinforcement Learning-Based Rule and Knowledge Graph Co-Optimization

1) *Markov Decision Process*: A decision is useful beyond its immediate step because acquiring a rule can unlock later repairs. We model this dependency as a finite-horizon Markov decision process. The 14-dimensional state is

$$s_t = [S(C_1), S(C_2), S(C_3), S(C_4), \text{Precision}, \text{Recall}, \text{Coverage}, \tilde{n}_{\text{iso}}, \tilde{n}_{\text{dup}}, \tilde{n}_{\text{invalid}}, \tilde{n}_{\text{dangling}}, \tilde{b}, \rho_{\text{valid}}, \rho_{\text{noisy}}]_t. \quad (8)$$

Here the four $\tilde{n}$ terms are current violation counts divided by their episode-initial counts, $\tilde{b}$ is the remaining decision budget, and the final two terms are fractions of valid and low-precision validators in the active rule set. All quantities are available from the current graph, validation inventory, and execution log; the state contains neither test labels nor the corruption manifest.

The eight discrete actions are

$$\mathcal{A} = \{a_{\text{iso}}, a_{\text{dup}}, a_{\text{rel}}, a_{\text{dang}}, a_{\text{del}}, a_{\text{aug}}, a_{\text{mix}}, a_{\text{prune}}\}. \quad (9)$$

The first four remove isolated noise nodes, deduplicate exact relations, restore schema-invalid relations, and remove dangling edges. The last four acquire a deletion-family validator, acquire an augmentation-family validator, acquire one validator from each family, and prune validators whose validation precision is below 0.5. A graph action is masked until its

validator is active or when no matching violation remains. This mask follows directly from the observable state and prevents no-op actions; it supplies no future reward information.

We optimize quality improvement while accounting for execution cost:

$$r_t = \Delta Q(G_t) + \lambda \Delta Q_{\text{online}}(R_t) - \mu C_{\text{call}} - \nu C_{\text{edit}} - \xi N_{\text{new}}, \quad (10)$$

where $\Delta Q(G_t) = Q(G_{t+1}) - Q(G_t)$ and the rule term is defined analogously. In the reported benchmark, $\lambda = 0.4$, $\mu = 0.004$, $\nu = 2 \times 10^{-5}$, and $\xi = 0.01$. An episode terminates when $Q(G) \geq 0.999$ and $Q_{\text{online}}(R) \geq 0.90$, or after 18 decisions.

2) *Double DQN*: The online network has architecture $14 \rightarrow 32 \rightarrow 16 \rightarrow 8$ with ReLU activations. Experience tuples $(s_t, a_t, r_t, s_{t+1}, d_t)$ are stored in replay memory. To reduce maximization bias, the online network selects the next action and the target network evaluates it:

$$y_t^{\text{DDQN}} = r_t + \gamma(1 - d_t)Q\left(s_{t+1}, \arg \max_{a' \in \mathcal{A}_{t+1}} Q(s_{t+1}, a'; \theta); \theta^-\right),$$

$$\mathcal{L}(\theta) = \mathbb{E} \left[\text{Huber} \left(Q(s_t, a_t; \theta) - y_t^{\text{DDQN}} \right) \right]. \quad (11)$$

The feasible set $\mathcal{A}_{t+1}$ applies the same observable action mask used during execution. The target parameters θ^- are copied from the online parameters every 100 optimizer steps.

We train with Adam at learning rate 2×10^{-3} , discount $\gamma = 0.95$, replay capacity 20,000, batch size 64, and a 128-transition warm-up. Exploration decreases linearly from 1.0 to 0.05 over 250 episodes. Each learned policy is trained independently for each of ten seeds. The training loop writes every evaluation transition, per-episode learning curve, configuration, and final checkpoint. Standard DQN uses the same implementation and hyperparameters, changing only the Bellman target to $\max_{a'} Q(s', a'; \theta^-)$.

3) *Execution Without Repeated LLM Calls*: Calling an LLM inside every training episode would conflate policy learning with model latency and sampling noise. We therefore train against executable validators and audit the archived rule-generation log, which contains 4,728 successful deletion calls and 4,728 successful augmentation calls. Training makes zero API calls. The API-backed extraction experiment remains an independent end-to-end test, while candidate-level and RuleTest-94 experiments assess the generated rule content. This separation makes the sequential-control result reproducible without presenting the controlled environment as a measure of open-world rule-generation accuracy.

D. LLM-Based Dual-Strategy Rule Generation

The RL agent in Section III-C treats rule generation as one class of actions—but *how* are rules actually generated when the agent selects a rule-generation action? This section answers that question. We propose a dual-strategy LLM mechanism that the RL agent invokes to produce new quality rules, which are then added to the rule set R and immediately reflected in the subsequent $Q_{\text{online}}(R)$ evaluation and reward.

1) *What Is a Quality Rule?*: Before describing the generation strategies, we clarify what a *quality rule* is, since both the RL agent (Section III-C) and the dual strategies operate on this shared concept.

Definition. A quality rule r is a first-order constraint over the knowledge graph G that, when violated, indicates a structural, logical, or semantic defect. Formally, we define four complementary rule types that cover the defect categories targeted in this work:

- 1) **Attribute Existence Rule.** An entity of type τ must possess a required property p :

$$r_{\text{attr}} : \text{type}(e) = \tau \implies \exists (e, p, v) \in G \quad (12)$$

- 2) **Relation Validity Rule.** A relation p between entities of types τ_s and τ_o must be semantically admissible:

$$r_{\text{rel}} : (s, p, o) \in G \implies \text{type}(s) \in \mathcal{T}_s^p \wedge \text{type}(o) \in \mathcal{T}_o^p \quad (13)$$

where $\mathcal{T}_s^p$ and $\mathcal{T}_o^p$ are the admissible subject/object type sets for predicate p .

- 3) **Value Constraint Rule.** A property value must satisfy a domain-specified constraint ϕ :

$$r_{\text{val}} : (e, p, v) \in G \implies \phi(v) \quad (14)$$

where ϕ can encode range constraints (e.g., $v > 0$ for monetary amounts), format constraints (e.g., valid date format), or enumeration constraints (e.g., $v \in \{\text{"Approved"}, \text{"Rejected"}\}$).

- 4) **Cross-Entity Ordering Rule.** A temporal or hierarchical ordering must hold between related entities:

$$r_{\text{ord}} : (e_1, p_1, v_1) \in G \wedge (e_2, p_2, v_2) \in G \wedge (e_1, \text{rel}, e_2) \in G \implies v_1 \prec v_2 \quad (15)$$

where $\prec$ denotes a partial order (e.g., temporal precedence, administrative hierarchy).

A triple or entity constitutes a **violation** of rule r if the antecedent holds but the consequent does not. A rule set $R = \{r_1, \dots, r_m\}$ partitions detectable defects into those flagged by at least one rule and those remaining undetected.

Scope. This unified definition subsumes the attribute-existence rules that dominate prior KG quality literature [30], [29] (corresponding to r_{attr}) while extending to relational validity (r_{rel}), value constraints (r_{val}), and cross-entity ordering (r_{ord}) that are essential for domain-specific KGs. The dual-strategy mechanism in Sections III-D2–III-D3 generates all four rule types automatically, guided by the LLM’s domain knowledge.

Examples. The following rules illustrate all four types across our evaluation domains:

Rule-set quality. The generation objective uses the online rule-quality score defined in Section III-B (Eq. 5):

$$Q_{\text{online}}(R) = 0.35 \text{ Precision}(R) + 0.35 \text{ Recall}(R) + 0.30 \text{ Coverage}(R). \quad (16)$$

TABLE II
Representative quality rules across types and domains.

Type	Domain	Formal Rule	Plain English
r_{attr}	Government	$\text{type}(e) = \text{PenaltyDecision} \Rightarrow (e, \text{issuedBy}, *) \in G$	Every penalty decision must name an issuing authority.
r_{rel}	Government	$(s, \text{manages}, o) \in G \Rightarrow \text{type}(s) \in \{\text{GovAgency}\}$	Only government agencies can hold a “manages” relation.
r_{val}	Finance	$(e, \text{penaltyAmount}, v) \in G \Rightarrow v > 0$	Penalty amounts must be strictly positive.
r_{ord}	Environment	$(e, \text{filingDate}, v_1) \in G \wedge (e, \text{closingDate}, v_2) \in G \Rightarrow v_1 \prec v_2$	Case filing date must precede closing date.

where the three metrics are as defined in Section III-B. The offline $\text{Unique}(R)$ metric is still reported in experiments, but it is not part of the generation objective. Formally, the rule generation objective maximises:

$$\max_R Q_{\text{online}}(R) \quad \text{s.t.} \quad \text{Precision}(R) \geq \theta_{\text{prec}} \quad (17)$$

where $\theta_{\text{prec}} = 0.95$ is a hard precision floor ensuring reliability.

Goal. Expert curation produces only a sparse, manually-chosen set of rules that covers a small fraction of entity types and often misses domain-specific edge cases. Our dual-strategy mechanism uses LLMs to automatically explore the rule space and discover a high-quality rule set that maximises $Q_{\text{online}}(R)$ while keeping $\text{Precision}(R)$ above a threshold.

Rule-family interpretation for ablation. For the executable rule-family ablation in Section IV-B4, we group validated rules by the dominant defect family they cover. Deletion-completion rules are primarily associated with procedural, missing-field, and specialist constraints, because masked completions reveal properties or steps that must be present. Augmentation-expansion rules are primarily associated with relation validity, hierarchy, type-direction, and boundary-case constraints, because synthetic variants expose alternative relation patterns and invalid structural combinations. The final system uses the union of both families; this grouping is used only for deterministic ablation and does not require every individual LLM-generated rule to carry a perfect strategy-provenance label.

2) *Strategy 1: Deletion Completion Strategy:* **Intuition:** The name reflects the two-step process: we first *delete* (mask) parts of the source text, then ask the LLM to *complete* the gaps. Whatever the LLM consistently fills in reveals an implicit constraint that it has learned from the domain—and that constraint becomes a quality rule.

Principle: An adversarial exploration method for discovering strongly constrained rules. We create masked text T_{masked} by randomly removing information from original text T . The LLM’s contextual reasoning to complete missing information exposes implicit rules it relies on.

Algorithm: Given source text T and current KG G :

Information-theoretic Interpretation: The deletion completion strategy discovers implicit constraints by maximizing Pointwise Mutual Information (PMI) between rules and context. Define:

$$\text{PMI}(r; C) = \log \frac{P(r, C)}{P(r) \cdot P(C)} \quad (18)$$

Algorithm 1 Deletion Completion Strategy

Require: Source text T , KG G , masking probability $p_{\text{mask}} = 0.15$, iterations $N = 10$

Ensure: Rule set R_{deletion}

```

1:  $R_{\text{deletion}} \leftarrow \emptyset$ 
2:  $\text{entity\_list} \leftarrow \text{ExtractEntities}(T)$ 
3: for  $i = 1$  to  $N$  do
4:    $T_{\text{masked}} \leftarrow T$ 
5:    $k \leftarrow \lceil p_{\text{mask}} \times |\text{entity\_list}| \rceil$ 
6:    $\text{masked\_entities} \leftarrow \text{RandomSample}(\text{entity\_list}, k)$ 
7:   for each  $e \in \text{masked\_entities}$  do
8:      $T_{\text{masked}} \leftarrow \text{Replace}(T_{\text{masked}}, e, [\text{MASK}])$ 
9:   end for
10:   $\hat{T} \leftarrow \text{LLM}(T_{\text{masked}}, G, \text{prompt\_completion})$ 
11:   $\Delta \leftarrow \text{Compare}(T, \hat{T}) \triangleright \text{Identify inferred constraints}$ 
12:  for each  $(e_{\text{type}}, p) \in \Delta$  do
13:    if  $\text{Frequency}(e_{\text{type}}, p) \geq \theta_{\text{freq}}$  then  $\triangleright \theta_{\text{freq}} = 0.7$ 
14:       $r \leftarrow \text{”Entities of type } e_{\text{type}} \text{ must have property } p\text{”}$ 
15:       $R_{\text{deletion}} \leftarrow R_{\text{deletion}} \cup \{r\}$ 
16:    end if
17:  end for
18: end for return  $R_{\text{deletion}}$ 

```

where r represents a candidate rule and C denotes the context. High PMI indicates strong statistical dependence: if the LLM consistently infers rule r from masked context C_{masked} , we have $\text{PMI}(r; C) \gg 0$, signaling a strongly constrained rule. Formally, given masking probability $p_{\text{mask}} = 0.15$ (following BERT’s masking strategy), we estimate rule confidence as:

$$\text{Confidence}(r) = \frac{1}{N} \sum_{i=1}^N \mathbb{I}\{\text{LLM}(C_{\text{masked}}^{(i)}) \text{ infers } r\} \quad (19)$$

where $N = 10$ is the number of masking iterations, and $\mathbb{I}\{\cdot\}$ is the indicator function. Rules with $\text{Confidence}(r) \geq \theta_{\text{freq}} = 0.7$ are extracted.

Example: Given text “The Environmental Protection Bureau issued a penalty decision”, if we mask “Environmental Protection Bureau” and the LLM consistently completes it with a government agency, we extract the rule: “penalty decisions must be issued by government agencies”.

3) *Strategy 2: Augmentation Expansion Strategy:* **Intuition:** Rather than masking existing text, we *augment* it—asking the LLM to generate varied versions of the original document. Mining rules across these synthetic variants *expands*

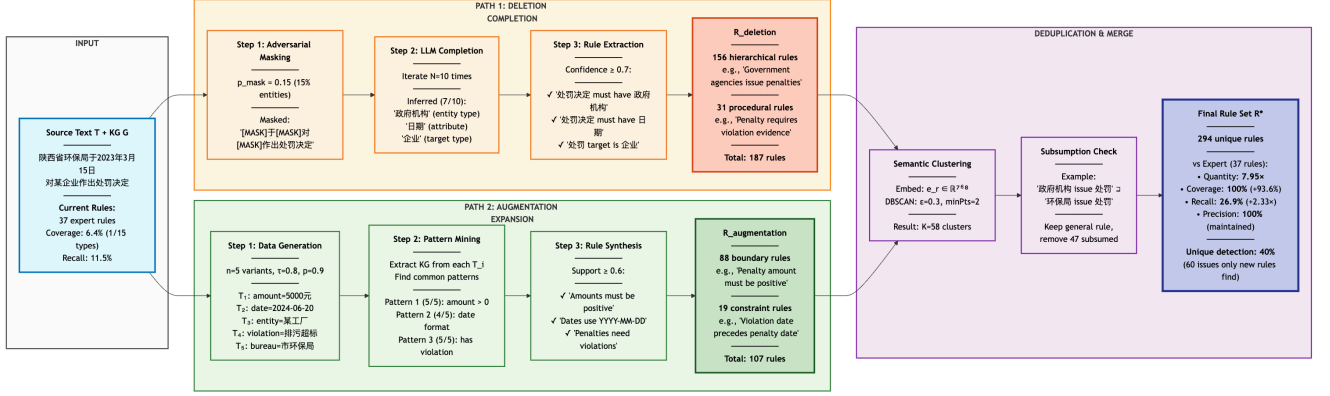


Fig. 2. Dual-Strategy Rule Generation: Complementary Coverage of Rule Space

Algorithm 2 Augmentation Expansion Strategy

Require: Source text T , augmentation count $n = 5$, temperature $\tau = 0.8$, nucleus sampling $p = 0.9$
Ensure: Rule set $R_{\text{augmentation}}$

```

1:  $R_{\text{augmentation}} \leftarrow \emptyset$ 
2:  $\text{augmented\_texts} \leftarrow \emptyset$ 
3: for  $i = 1$  to  $n$  do
4:    $T_i^{\text{aug}} \leftarrow \text{LLM}(T, \text{prompt\_augment}, \tau, p)$ 
5:    $\text{augmented\_texts} \leftarrow \text{augmented\_texts} \cup \{T_i^{\text{aug}}\}$ 
6: end for
7:  $G_{\text{original}} \leftarrow \text{ExtractKG}(T)$ 
8: for each  $T_i^{\text{aug}} \in \text{augmented\_texts}$  do
9:    $G_i \leftarrow \text{ExtractKG}(T_i^{\text{aug}})$ 
10: end for
11:  $\text{patterns} \leftarrow \text{MineCommonPatterns}(\{G_1, \dots, G_n\}, G_{\text{original}})$ 
12: for each  $\text{pattern} \in \text{patterns}$  do
13:   if  $\text{Support}(\text{pattern}) \geq \theta_{\text{support}}$  then  $\triangleright \theta_{\text{support}} = 0.6$ 
14:      $r \leftarrow \text{SynthesizeRule}(\text{pattern})$ 
15:      $R_{\text{augmentation}} \leftarrow R_{\text{augmentation}} \cup \{r\}$ 
16:   end if
17: end for return  $R_{\text{augmentation}}$ 

```

coverage to boundary cases and rare scenarios that the single original text does not expose.

Principle: An exploratory generation method for discovering edge-case rules. We use LLM’s generation capability to produce supplementary clauses or edge cases for original text T (resulting in $T_{\text{augmented}}$), enabling rule mining on new virtual samples.

Algorithm: Given source text T :

Information-theoretic Interpretation: The augmentation expansion strategy provides robust rule estimation through distributional sampling. A single text T represents only one realization from the true data distribution $\mathcal{D}$. LLM-driven augmentation samples from the neighborhood $\mathcal{N}(T)$ centered at T , enabling estimation of the posterior rule probability:

$$P(r \mid \mathcal{D}) \approx \mathbb{E}_{T' \sim \mathcal{N}(T)}[P(r \mid T')] \approx \frac{1}{n} \sum_{i=1}^n P(r \mid T_i^{\text{aug}}) \quad (20)$$

where $T_i^{\text{aug}} \sim \text{LLM}(T; \tau = 0.8, p = 0.9)$ are augmented samples generated with temperature τ and nucleus sampling parameter p . The rule support is computed as:

$$\text{Support}(r) = \frac{|\{i : r \in \text{Rules}(G_i)\}|}{n} \quad (21)$$

where $G_i = \text{ExtractKG}(T_i^{\text{aug}})$ is the knowledge graph extracted from the i -th augmented text. Rules with $\text{Support}(r) \geq \theta_{\text{support}} = 0.6$ are retained, ensuring robustness across diverse variations.

Example: Given text about a penalty case, augmentation might generate variations with different penalty amounts, dates, or violation types. Rules extracted from these variations capture boundary conditions (e.g., “penalty amounts must be positive”, “violation dates must precede penalty dates”).

4) **Rule Materialization and Validation:** The two strategies above produce natural-language constraints. To make the generated rules executable and reproducible, we convert each candidate into a structured rule record before adding it to the rule set. The materialization pipeline contains four deterministic steps:

- 1) **Parsing and normalization.** Each LLM-returned constraint is parsed into a JSON-like record with fields type, if, then, source, and confidence. The type field must be one of the four rule types defined in Section III-D1: `attr_required`, `rel_type`, `value_constraint`, or `order_constraint`. Surface variants such as “issuer”, “issuing department”, and “issued by” are normalized to the canonical predicate used by the KG schema.
- 2) **Threshold filtering.** Deletion-completion candidates are retained only when their empirical confidence satisfies $\text{Confidence}(r) \geq 0.7$. Augmentation-expansion candidates are retained only when their support satisfies $\text{Support}(r) \geq 0.6$. Candidates that cannot be mapped to an existing entity type or predicate are discarded.
- 3) **Validation on clean and defective cases.** Before a candidate enters R , it is executed on a small validation split containing both clean triples and injected defects. Rules that trigger false positives on clean cases are rejected unless their predicate mapping can be repaired deterministically. This step enforces the precision floor $\text{Precision}(R) \geq \theta_{\text{prec}}$ used in Eq. 17.
- 4) **Executable compilation.** Validated records are compiled into rule checkers: attribute rules become missing-property checks, relation rules become subject/object type checks, value rules become literal validators, and ordering rules become pairwise comparison checks over

normalized temporal or hierarchical values.

Running example. Suppose the source text states that a municipal environmental bureau issued an administrative penalty decision, and the extracted KG contains:

(*PenaltyDecision_01*, *penaltyAmount*, -5000).

Augmentation expansion repeatedly produces plausible penalty cases with positive monetary amounts. The LLM returns the constraint "penalty amount must be greater than zero", which is materialized as:

```
{
  "type": "value_constraint",
  "if": {"entity_type": "PenaltyDecision",
        "property": "penaltyAmount"},
  "then": {"operator": ">", "value": 0},
  "source": "augmentation",
  "confidence": 0.80
}
```

After validation, the compiled checker applies the predicate $\phi(v) : v > 0$ to all *PenaltyDecision* amount values. It flags the triple above as a violation, updates the rule-quality statistics used by $Q_{\text{online}}(R)$, and provides the RL agent with a reward signal if the subsequent graph-enhancement action repairs the value.

5) *Rule Aggregation and Deduplication*: The final rule set combines both strategies with deduplication to remove redundant rules while preserving coverage:

$$R^* = \text{Deduplicate}(R_{\text{deletion}} \cup R_{\text{augmentation}}) \quad (22)$$

Deduplication Algorithm:

Step 1 - Semantic Clustering: Embed each rule as $\mathbf{e}_r = \text{f}_{\text{embed}}(\text{rule_text})$ using sentence transformers, where $\mathbf{e}_r \in \mathbb{R}^d$ ($d = 768$). Apply DBSCAN clustering with:

$$\text{dist}(r_i, r_j) = 1 - \cos(\mathbf{e}_{r_i}, \mathbf{e}_{r_j}) = 1 - \frac{\mathbf{e}_{r_i} \cdot \mathbf{e}_{r_j}}{\|\mathbf{e}_{r_i}\| \|\mathbf{e}_{r_j}\|} \quad (23)$$

with radius $\epsilon = 0.3$ and $\text{minPts} = 2$.

Step 2 - Subsumption Check: Within each cluster C_k , identify subsumption relationships. Rule r_i subsumes r_j (denoted $r_i \sqsupseteq r_j$) if:

$$\text{Entities}(r_j) \subset \text{Entities}(r_i) \wedge \text{Semantics}(r_i) \implies \text{Semantics}(r_j) \quad (24)$$

For example, "government agencies issue penalties" $\sqsupseteq$ "environmental bureaus issue penalties" (since environmental bureaus $\subset$ government agencies).

Step 3 - Rule Selection: For each cluster C_k , construct the subsumption partial order and retain:

$$R_k^* = \{r \in C_k : \nexists r' \in C_k, r' \sqsupseteq r \wedge r' \neq r\} \quad (25)$$

This keeps maximally general rules while preserving specific rules that add unique constraints not covered by generalizations.

The final deduplicated rule set is $R^* = \bigcup_{k=1}^K R_k^*$ where K is the number of clusters.

This dual-strategy method transforms rule generation from manual writing to LLM-driven automatic exploration, addressing insufficient coverage and high maintenance costs.

The deletion strategy discovers implicit constraints through adversarial masking, while the augmentation strategy explores boundary cases through data generation, providing complementary coverage of the rule space.

6) Implementation Details and Reproducibility: LLM Configuration:

- *Rule-generation model*: Qwen3-32B accessed via API for deletion completion and augmentation expansion
- *API extraction benchmark model*: Qwen3.8-27B-no-thinking, used only for the TNEWS/CLUE API-backed extraction benchmark in Section IV-F
- *Temperature*: $\tau = 0.3$ for completion tasks (deterministic), $\tau = 0.8$ for augmentation (diverse)
- *Max tokens*: 2048 for completion, 1024 for augmentation
- *Top-p sampling*: $p = 0.9$ for augmentation to balance diversity and quality

Prompts:

Deletion Completion Prompt:

Given the following text with masked entities [MASK], complete the missing information based on the context and knowledge graph structure:

Text: {masked_text}

Knowledge Graph: {kg_triples}

Complete the [MASK] tokens with the most appropriate entities or relations. Provide only the completed text.

Augmentation Expansion Prompt:

Generate 5 semantically similar but structurally different versions of the following text. Vary the specific details (amounts, dates, entities) while preserving the core semantic relationships:

Original Text: {text}

Generate variations that explore edge cases and boundary conditions.

Computational Cost:

- *Archived rule generation*: 9,456 successful API calls over 4,728 government-domain documents (4,728 calls per strategy)
- *RL benchmark*: 1,278.99 seconds on CPU for ten DQN and ten Double-DQN training runs of 250 episodes each
- *RL API calls*: zero; training reuses executable validators and reports model-call cost in the reward

Reproducibility Statement: The repository includes the prompts, RuleTest-94 cases, aggregated rule set, candidate logs, aggregation and validation code, RL runner, twenty learned checkpoints, all evaluation transitions, per-episode training histories, fixed seeds, and statistical-test inputs. The archived rule-generation script records Qwen3-32B as its default generation backbone. The independent extraction stress test uses Qwen3.8-27B-no-thinking; it does not generate the rule set reported in the rule experiments.

TABLE III
Comparison of Rule Generation Quantity

Rule Type	Expert	Auto-gen.	Improv.
Entity type	7	77	11.00×
Relation type	9	80	8.89×
Other type constraints	21	15	0.71×
Hierarchical	0	31	Exclusive
Procedural	0	88	Exclusive
Total	37	291	7.86×

IV. Experiments

A. Experimental Setup

1) *Evaluation Questions*: The evaluation addresses five questions:

- **RQ1: Rule-space expansion.** How many distinct rule candidates do the two generation strategies contribute?
- **RQ2: Executable detection.** Do the final rule families detect complementary defects on RuleTest-94?
- **RQ3: Adaptive optimization.** Does a learned policy outperform fixed and sequential schedules under identical graph transitions and budgets?
- **RQ4: Constraint checking.** What does generated validation add beyond structural SHACL-style checks?
- **RQ5: End-to-end robustness.** Does the quality-control layer improve deterministic and API-extracted KGs?

2) *Evaluation Pools and Scope*: RuleTest-94 is a designed executable test suite, not a prevalence sample. It contains 30 clean cases and 64 defective cases: 10 hierarchy reversals, 10 absurd relations, 10 reversed-supervision relations, 30 procedural or missing-field defects, and 4 specialist constraints. The clean cases support false-positive measurement. Each detector writes a prediction for every case, allowing the confusion matrices to be reconstructed from the released CSV files.

The candidate-level experiment replays 9,456 successful stored generation responses: 4,728 deletion-completion calls and 4,728 augmentation-expansion calls. Its unit is a canonicalized candidate rule, so candidate counts must not be interpreted as detection accuracy. The RL benchmark uses a separate 450-document TNEWS/CLUE graph with controlled graph mutations. The deterministic external benchmark uses 3,000 documents, and the API extraction benchmark uses 45 titles. Keeping these pools separate prevents the designed RuleTest suite or controlled corruption manifest from being presented as naturally occurring test data.

3) *Runtime and Reproducibility Settings*: All deterministic scripts use fixed recorded seeds. The RL comparison uses ten paired evaluation seeds and ten independently trained models per learned method. It archives checkpoints, all evaluation transitions, and per-episode histories. The API benchmark makes 45 calls to Qwen3.8-27B-no-thinking through the configured OpenAI-compatible endpoint with proxy inheritance disabled; parse or request failures remain in the denominator. Rule candidates are materialized through the validation pipeline in Section III-D4.

B. Rule Generation Results

TABLE IV
Executable rule detection on RuleTest-94.

Rule set	Precision	Recall	F1	Family cov.
Expert structural	1.000	0.469	0.638	0.600
System union	1.000	1.000	1.000	1.000

TABLE V
Candidate-level ablation of deletion and augmentation strategies on stored government-domain generation logs.

Strategy	Calls	Cand. Rules	Cov. vs Dual	Unique
Deletion only	20	285	0.483	262
Augmentation only	20	328	0.556	305
Dual strategy	40	590	1.000	567

1) *Quantity Comparison: Key Finding*: The evaluated generated set contains 291 rules versus 37 expert rules (7.86×), including 31 hierarchical and 88 procedural rules absent from the expert set.

2) *Executable Detection on RuleTest-94*: The expert structural detector identifies 30 of 64 defects and makes no false-positive prediction on the 30 clean cases. The system union identifies all 64 defects with no false positives on this designed suite. Its additional 34 detections are procedural, missing-field, or specialist cases, so its unique detection capability relative to the expert detector is $34/64 = 0.531$. The 95% Wilson interval for system precision uses 64 predicted positives and is $[0.943, 1.000]$. These values establish executable suite coverage; they do not imply perfect open-world precision.

3) *Candidate-Level Dual-Strategy Ablation*: To verify that deletion completion and augmentation expansion contribute complementary rule candidates, we re-aggregate the stored LLM outputs by strategy before final validation and deduplication. This analysis does not replace executable detection on RuleTest-94; it measures how much of the candidate rule space each strategy contributes.

The two strategies overlap on only 23 candidate rules in this controlled 20-item log, while deletion contributes 262 unique candidates and augmentation contributes 305. The dual strategy therefore yields 79.9% more candidates than the stronger single strategy. Running the same aggregation on the full available government-domain log (4,728 items per strategy) gives the same pattern: deletion produces 32,955 candidates, augmentation produces 41,683, and the dual strategy produces 71,656 candidates, a 71.9% gain over the stronger single strategy. This supports the complementarity claim at the generation stage; final rule precision still depends on the validation and filtering pipeline described in Section III-D4.

4) *Rule-Family Final Ablation on RuleTest-94*: The candidate-level ablation above measures generated rule diversity before final validation. To test whether the two strategy families also provide complementary executable detection coverage, we run a deterministic rule-family ablation on RuleTest-94. Deletion-family rules cover procedural, missing-field, and specialist constraints, while augmentation-family rules cover structural relation, hierarchy, and type-direction constraints. This experiment should be interpreted as a final

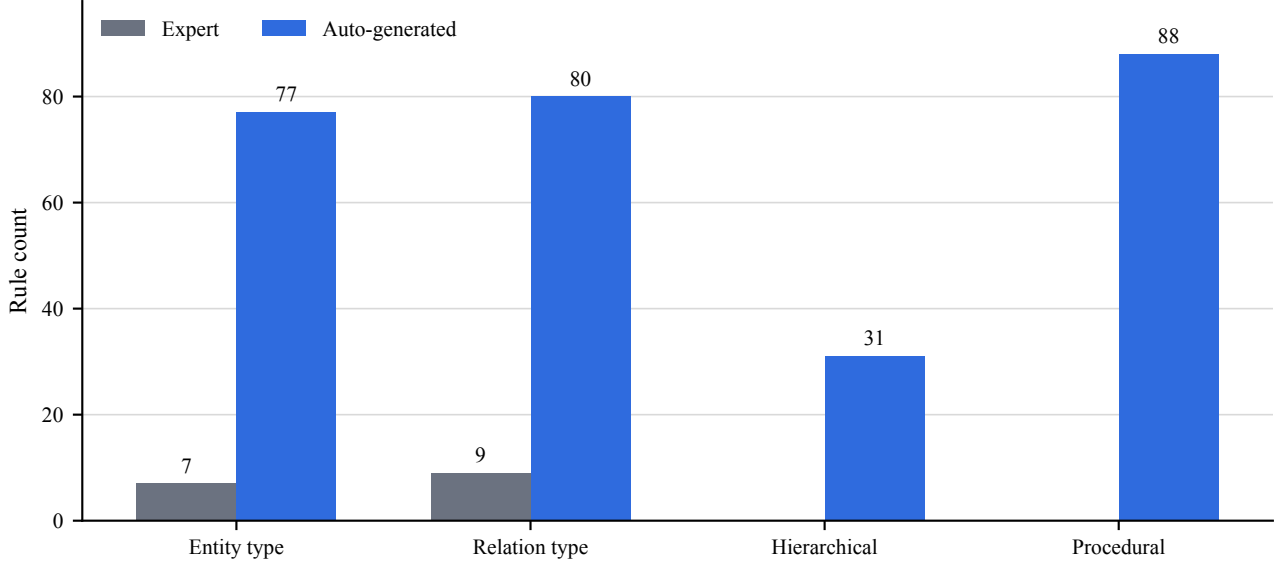


Fig. 3. Comparison of Rule Generation Quantity

TABLE VI
Rule-family final ablation on RuleTest-94.

Strategy	Precision	Recall	F1	Fam. Cov.
Deletion-family only	1.000	0.531	0.694	0.400
Augmentation-family only	1.000	0.469	0.638	0.600
Dual strategy	1.000	1.000	1.000	1.000

rule-family ablation, not as per-rule provenance labeling for every generated LLM rule.

The deletion-family detector captures 34 of 64 defective cases, while the augmentation-family detector captures a different set of 30 structural cases. Their union detects all 64 defective cases without false positives on the 30 clean cases in RuleTest-94. This result strengthens the complementarity claim: the two strategies are not merely producing more candidates, but cover different executable defect families. We use this result as supporting evidence for strategy complementarity; the detection comparison remains the executable RuleTest-94 evaluation in Table IV.

C. RL Co-optimization on Executable Graph Transitions

1) *Controlled Environment and Protocol*: The policy benchmark uses 450 balanced TNEWS/CLUE documents (30 from each of 15 categories) to construct a clean graph with 873 nodes and 1,083 relations. For each evaluation seed, the environment injects seed-dependent amounts of four defects: isolated nodes, exact duplicate relations, schema-invalid relations, and dangling edges. Online rule metrics use a fixed 180-case validation inventory containing 30 positives for each defect family and 60 clean cases; the four valid modules each detect 27 of 30 family positives plus two clean cases, while three low-precision modules exercise the pruning action. This inventory is part of the controlled environment and is separate from RuleTest-94. Each action changes the graph or active rule

set, after which all four graph metrics and three rule metrics are recomputed. The policy cannot observe the corruption manifest. Relation restoration consults the manifest only after the invalid-relation validator has fired, analogous to using a trusted source record for repair.

We train standard DQN and Double DQN independently for ten seeds, using 250 episodes per seed. Evaluation uses the same ten paired scenario seeds for every method. Besides random selection, we implement graph-only, rule-only, alternating, rule-first-then-fix, and fix-first-then-rule schedules. A one-step myopic policy clones the current environment and executes every feasible action before choosing; because this gives it a transition model unavailable to the other methods, we report it as a model-informed upper-bound diagnostic rather than as the primary baseline. Normalized joint quality is

$$\bar{Q} = \frac{Q(G) + 0.4Q_{\text{online}}(R)}{1.4}, \quad (26)$$

and AUC is the mean of $\bar{Q}$ over the decision trajectory. Convergence is the first step at which $\bar{Q} \geq 0.98$; a value of 19 denotes failure within the 18-step budget.

2) *Policy Comparison*: Double DQN improves final quality by 0.0065 and AUC by 0.0166 over the strongest non-oracle fixed schedule, rule-first-then-fix. Across the ten paired seeds, the 95% paired-bootstrap intervals are [0.0058, 0.0071] for final quality and [0.0156, 0.0178] for AUC; both one-sided Wilcoxon tests give $p = 0.000977$. The learned policy also reduces rule-generation calls from 8.0 to 4.0. DQN and Double DQN have essentially the same mean performance; Double DQN has lower AUC variability (0.0021 versus 0.0041). We therefore attribute the gain to learned state-conditioned control and do not claim a significant mean advantage of Double DQN over standard DQN.

The myopic model-informed policy remains slightly stronger in AUC. This is expected because it evaluates every one-step transition before acting. Double DQN approaches that

TABLE VII
Co-optimization results on executable graph transitions (mean $\pm$ standard deviation over 10 paired seeds).

Policy	Final $\bar{Q}$	Trajectory AUC	Steps to 0.98	Calls	Edits
Random	0.9535 $\pm$ 0.0266	0.8829 $\pm$ 0.0318	19.0	8.3	151.4
Enhancement only	0.6785 $\pm$ 0.0036	0.6785 $\pm$ 0.0036	19.0	0.0	0.0
Rule only	0.9373 $\pm$ 0.0036	0.9225 $\pm$ 0.0036	19.0	18.0	24.0
Alternating	0.9634 $\pm$ 0.0007	0.9261 $\pm$ 0.0020	19.0	12.0	215.0
Rule first, then fix	0.9747 $\pm$ 0.0004	0.9365 $\pm$ 0.0024	19.0	8.0	232.7
Fix first, then rule	0.9330 $\pm$ 0.0036	0.8204 $\pm$ 0.0036	19.0	12.0	13.0
Myopic model-informed	0.9825 $\pm$ 0.0001	0.9576 $\pm$ 0.0009	13.2	4.0	242.2
DQN	0.9813 $\pm$ 0.0017	0.9530 $\pm$ 0.0041	16.8	4.1	233.8
Double DQN	0.9812 $\pm$ 0.0013	0.9531 $\pm$ 0.0021	17.0	4.0	234.9

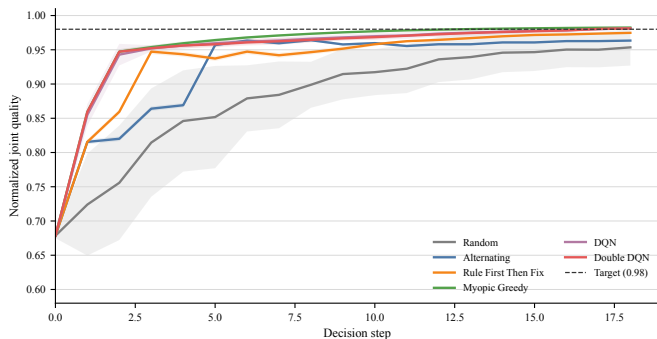


Fig. 4. Evaluation trajectories over ten paired seeds. Lines show means and shaded bands show one standard deviation. The myopic curve is a model-informed upper bound.

upper bound without access to the transition function: its final-quality gap is 0.0013 and its AUC gap is 0.0045.

3) *Learned Action Use and Reproducibility*: The Double DQN policy selects mixed rule generation exactly twice per run on average, activating the four high-precision validators with four accounted calls. It then allocates the remaining decisions according to the unresolved violation counts: 3.3 isolation cleanups, 5.2 deduplication steps, 4.0 relation repairs, and 3.5 dangling-edge cleanups per run. This ordering is state-dependent because corruption rates differ by seed. It does not repeatedly call the rule generator after all four validators are available.

The experiment archives 2,500 training episodes per learned method, 1,616 evaluation transitions across all policies, twenty model checkpoints, per-seed metrics, and the exact statistical test inputs. Training makes no API calls. The stored dual-strategy generation log contains 9,456 successful calls (4,728 per strategy), which are reused for the separate rule-content analyses rather than regenerated inside RL episodes.

D. Comparison with Constraint-Checking Baseline

We compare with a deterministic constraint-checking baseline implemented with SHACL-style RDF validation. This baseline receives deduplicated RDF triples and checks schema-conformance constraints, but it does not access the source text, LLM prompts, or generated procedural/value rules. The experiment therefore measures whether conventional graph-shape validation is sufficient for our quality-control task.

TABLE VIII
SHACL-style structural validation baseline on three domains.

Domain	Raw Triples	RDF Dedup.	Removed	Q
Gov. Affairs	13,892	10,771	0	79.87
Finance	2,264	539	0	70.42
Env.	2,345	378	0	65.25
Average	6,167	3,896	0	71.85

TABLE IX
TNEWS/CLUE local end-to-end KG quality benchmark.

Stage	Nodes	Relations	Invalid	Q
Clean extraction	10,871	20,827	0.000	99.89
Corrupted KG	11,414	22,909	0.050	94.93
Repaired KG	10,822	19,799	0.000	100.00

The SHACL-style baseline conforms after RDF deduplication and removes no additional triples, because its schema-level constraints cannot express many of the source-text-dependent, value-range, temporal, and procedural defects targeted by our rule set. Its average quality score is 71.85, limited by redundancy, semantic soundness, and residual logical checks rather than SHACL conformance. This supports the need for generated domain rules beyond static shape validation.

E. External Benchmark Platform

To test whether the quality-enhancement pipeline behaves robustly outside the three in-domain regulatory datasets, we built a local end-to-end benchmark platform using two existing benchmark-style resources in the repository. First, we use a TNEWS/CLUE-style Chinese news dataset as an out-of-domain text source. A deterministic extractor constructs a document-category-entity KG from 3,000 balanced news titles across 15 categories; we then inject duplicate triples, invalid relations, and isolated nodes, and run the local repair pipeline. Second, we use the 94-case rule-test set as a labeled rule-detection benchmark. This benchmark is fully local and makes no LLM/API calls, so it evaluates KG quality control and rule execution rather than LLM extraction quality.

The repair pipeline removes injected invalid relations, duplicate triples, and isolated noise nodes, improving Q from 94.93 to 100.00 and recovering the full injected quality gap under

TABLE X
RuleTest-94 local rule-detection benchmark.

Detector	Precision	Recall	F1	Accuracy
Expert structural rules	1.000	0.469	0.638	0.638
System rule detector	1.000	1.000	1.000	1.000

TABLE XI
API-backed LLM extraction benchmark on TNEWS/CLUE titles.

Metric	Raw LLM KG	Repaired KG
Parse success	1.000	—
Category accuracy	0.556	0.556
Keyword recall	0.178	0.178
Documents with triples	1.000	1.000
Avg. triples / doc	3.58	3.58
Invalid triple rate	0.000	0.000
Duplicate triple rate	0.000	0.000
KG quality score	100.00	100.00

this deterministic benchmark. The full report is generated by `exps/external_benchmark_runner.py`.

The expert structural baseline detects hierarchy reversal, absurd relations, and reversed supervision, but misses procedural, missing-field, and specialist-rule defects. The system rule detector covers these additional categories and passes all 64 defective cases in RuleTest-94 without false positives. Because RuleTest-94 is a designed local test suite, this result should be interpreted as benchmark-suite coverage rather than universal real-world precision.

F. API-Backed LLM Extraction Benchmark

The local TNEWS/CLUE benchmark above isolates the quality-control layer under deterministic extraction. To additionally test the full extraction path, we run an API-backed LLM extraction benchmark on 45 TNEWS/CLUE titles covering 15 categories. The model is Qwen3.8-27B-no-thinking. TNEWS category labels are used as silver labels for category inference, and provided title keywords are used as weak silver mentions for entity coverage. This is not a gold triple-level benchmark; it is a stress test for parsing, extraction coverage, and post-extraction repair under real model outputs.

The 45 calls completed in 1,190.90 seconds and all responses parsed successfully. Qwen3.8-27B-no-thinking followed the allowed schema on every sampled title, so the conservative repair layer made no change: raw and repaired structural quality are both 100.00. This no-op result is informative because it shows that the current model does not need structural filtering on this sample. Category accuracy is 0.556 and weak keyword recall is 0.178, indicating that semantic extraction remains the larger bottleneck; deterministic structural checks cannot repair omitted or misclassified content.

G. Runtime and Cost Accounting

We report runtime and call accounting for the benchmark runs whose execution logs are available in the repository. Table XII separates deterministic local experiments from API-dependent runs, because their variance and cost profiles are different.

TABLE XII
Runtime and model-call accounting for benchmark components.

Component	Items	Calls	Runtime / Note
TNEWS local repair	3,000 docs	0	0.113 s
RuleTest-94	94 cases	0	local rule execution
API extraction	45 docs	45	1,190.90 s
RL co-optimization	450 docs	0	1,278.99 s (CPU)
Dual-strategy gov full	4,728 docs	9,456	stored LLM logs

The local repair and rule-detection benchmarks are cheap and deterministic, making them suitable for regression testing. In contrast, dual-strategy rule generation scales linearly with the number of source documents and strategy calls. This motivates caching generated candidates and using executable replay during policy training rather than repeatedly invoking an LLM in every episode.

H. Limitations and Failure Boundaries

The experiments support a narrower claim than unrestricted deployment. First, RuleTest-94 is a designed suite whose executable families are known in advance. Its perfect system-union score demonstrates implementation coverage of those five families, but it cannot estimate defect prevalence or precision on arbitrary production graphs. The Wilson interval and the explicit 30 clean cases quantify sampling uncertainty only within that suite.

Second, the RL environment uses controlled corruption so that every graph transition can be checked exactly. This is useful for testing sequential control, replay, and Double-DQN learning, but the transition distribution is simpler than errors produced by an unconstrained extraction model. The model-informed myopic policy also shows that a known one-step transition model remains a strong upper bound. Evaluation on logged real repair trajectories is an important next step.

Third, the API extraction benchmark uses TNEWS categories and title keywords as silver labels. Category accuracy and keyword recall therefore measure weak extraction indicators, not gold triple correctness. The repair layer intentionally filters malformed or schema-invalid triples and cannot recover facts that the extractor omitted; improved structural quality can consequently coexist with unchanged or lower extraction recall.

Finally, the archived candidate logs provide strategy-level provenance, but the 291-rule comparison artifact does not preserve an independently adjudicated provenance label and correctness judgment for every rule. We therefore report candidate-level complementarity and an executable family ablation separately. Claims about cross-domain transfer and direct AMIE/RuDiK superiority require held-out domain labels and per-case baseline predictions; because those artifacts are absent, we do not report the earlier exploratory numbers as experimental results.

V. Conclusion and Future Work

We presented a knowledge-graph and rule co-optimization framework in which rule acquisition unlocks executable graph

repairs and the value of further rule generation depends on the current graph state. The resulting implementation is a genuine Double-DQN system with replay, a target network, masked feasible actions, and archived state transitions.

On ten paired controlled-corruption runs, Double DQN reaches normalized final quality 0.9812 ± 0.0013 and AUC 0.9531 ± 0.0021 . It improves final quality by 0.0065 and AUC by 0.0166 over the strongest non-oracle fixed schedule, with $p = 0.000977$ in both paired one-sided Wilcoxon tests, while reducing rule-generation calls from eight to four. Standard DQN has a similar mean, so the evidence supports learned state-conditioned control; it does not establish a mean advantage of Double DQN over DQN. The model-informed myopic policy remains a slightly stronger upper bound.

The rule experiments show complementary behavior at two levels. In the full archived log, the dual strategy contributes 71.9% more canonicalized candidates than the stronger single strategy. On the designed RuleTest-94 suite, deletion and augmentation families detect disjoint groups and their union covers all included cases. This suite result establishes executable coverage of known families rather than universal precision.

The next evaluation step is to replace controlled mutations with independently collected real repair trajectories and to create held-out domain test sets with per-case predictions from rule-mining baselines. Gold triple annotations are also needed for the API extraction benchmark, whose current category and keyword measures are silver indicators. These additions would test whether the learned scheduling benefit and generated constraints transfer beyond the controlled environment used here.

VI. Complete Prompts

This appendix provides the verbatim prompts used in the main dual-strategy rule-generation calls, enabling reproducibility of the rule generation module. The rule-generation backend is Qwen3-32B accessed via API; the same prompts were used across all three domains without modification. The separate API-backed extraction benchmark in Section IV-F uses Qwen3.8-27B-no-thinking only for testing the extraction path.

A. Deletion Completion Prompt

The following prompt is sent as a single user message. The {masked_text} and {kg_triples} placeholders are filled with the domain-specific masked document and the current KG triples (serialized as subject–predicate–object lines) respectively.

You are a domain expert assistant for knowledge graph quality analysis.

Given the following text with some entities replaced by [MASK] tokens, and the current knowledge graph triples extracted from the original text, your task is to:

1. Complete the [MASK] tokens with the most appropriate entities or relations based on the context and knowledge graph structure.
2. Identify any implicit constraints that must hold for the completed

entities to be semantically valid.

3. Format each identified constraint as a rule:

"Entities of type <TYPE> must have property <PROPERTY>"
or "Relation <REL> requires subject type <TYPE_S> and object type <TYPE_O>"
or "Property <PROP> of type <TYPE> must satisfy: <CONDITION>"

Text with masked entities:

{masked_text}

Current knowledge graph triples:

{kg_triples}

Output format:

COMPLETED_TEXT: <the text with [MASK] filled in>

CONSTRAINTS:

- <constraint 1>

- <constraint 2>

...

B. Augmentation Expansion Prompt

The following prompt is sent as a single user message. The {text} placeholder is filled with the original source document.

You are a domain expert assistant specializing in regulatory and administrative documents.

Generate 5 semantically similar but structurally diverse versions of the following text. Each version should:

1. Vary specific details: amounts, dates, entity names, violation types
2. Preserve core semantic relationships and domain constraints
3. Explore realistic boundary conditions and edge cases that might occur in practice (e.g., missing fields, borderline values, unusual but valid configurations)
4. Remain factually plausible within the domain

Original text:

{text}

For each generated version, also list any domain constraints that the variation reveals (e.g., "penalty amount must be positive", "filing date must precede closing date").

Output format for each variation (repeat 5 times):

VARIATION <N>:

TEXT: <the generated text>

REVEALED_CONSTRAINTS:

- <constraint 1>

- <constraint 2>

C. Prompt Design Rationale

Both prompts share two design principles:

- 1) **Explicit constraint elicitation:** Rather than asking the LLM to generate rules abstractly, we embed constraint

1. Complete the [MASK] tokens with the most appropriate entities or relations based on the context and knowledge graph structure. grounds the output in the specific text and reduces hallucination.
2. Identify any implicit constraints that must hold for the completed

TABLE XIII
Executable cases in RuleTest-94.

Case family	Count
Clean/pass	30
Hierarchy reversal	10
Absurd relation	10
Reverse supervision	10
Procedural or missing field	30
Specialist constraint	4
Total	94

- 2) **Structured output format:** The CONSTRAINTS: / REVEALED_CONSTRAINTS: section enforces that each LLM call returns parseable rule candidates, enabling deterministic downstream processing (frequency filtering, DBSCAN clustering).

Temperature settings: $\tau = 0.3$ for deletion completion (near-deterministic, maximises consistency across $N = 10$ masking iterations); $\tau = 0.8$ for augmentation expansion (higher diversity, sampling from $\mathcal{N}(T)$).

VII. RuleTest-94 Composition and Uncertainty

The benchmark contains 64 defective cases and 30 clean cases. For the system union, all 64 defective cases are predicted positive and all 30 clean cases are predicted negative. Precision is therefore $64/64 = 1.0$, recall is $64/64 = 1.0$, and specificity is $30/30 = 1.0$. For a boundary proportion, we report the Wilson interval rather than the degenerate Wald interval:

$$CI_{\text{Wilson}} = \frac{\hat{p} + z^2/(2n) \pm z\sqrt{\hat{p}(1-\hat{p})/n + z^2/(4n^2)}}{1 + z^2/n}. \quad (27)$$

With $n = 64$, $\hat{p} = 1$, and $z = 1.96$, the 95% precision interval is $[0.943, 1.000]$. The clean-case specificity interval, using $n = 30$, is $[0.886, 1.000]$.

VIII. RL Implementation and Archived Artifacts

The policy state contains fourteen values in this order: four graph-quality components, rule precision/recall/coverage, four normalized violation counts, remaining budget, valid-rule fraction, and noisy-rule fraction. The action indices are fixed as follows:

- 1) isolate cleanup;
- 2) exact relation deduplication;
- 3) schema-invalid relation repair;
- 4) dangling-edge cleanup;
- 5) deletion-family rule acquisition;
- 6) augmentation-family rule acquisition;
- 7) mixed rule acquisition;
- 8) low-precision rule pruning.

The Q-network is $\mathbb{R}^{14} \rightarrow \mathbb{R}^{32} \rightarrow \mathbb{R}^{16} \rightarrow \mathbb{R}^8$. Adam uses learning rate 2×10^{-3} ; $\gamma = 0.95$; replay capacity is 20,000; batch size is 64; learning starts after 128 transitions; and target parameters are copied every 100 optimizer steps. Exploration decreases linearly from 1.0 to 0.05 over 250 episodes.

For every reported evaluation step, transitions.csv records the state, action, reward, next state, terminal flag, graph quality, rule quality, calls, edits, rule count, and residual defect counts. per_seed_results.csv contains the paired inputs to the Wilcoxon and bootstrap analyses. The checkpoint directory contains ten DQN and ten Double-DQN models, while training_history.csv contains 5,000 episode summaries.

A. Rule Representation Format

To make rule generation and evaluation deterministic, all rules are stored as structured records (JSON). Each record contains:

- id: unique rule identifier
- domain: domain name (government/finance/environment)
- type: one of attr_required, rel_type, value_constraint, order_constraint
- if: rule condition (entity type / relation)
- then: required property/constraint (property name, allowed types, numeric bounds, temporal predicate, etc.)
- source: deletion, augmentation, or mix
- confidence: empirical confidence estimated from repeated LLM calls / frequency filtering

Example (attribute-required):

```
{
  "id": "gov_attr_issuedBy_001",
  "domain": "government",
  "type": "attr_required",
  "if": {"entity_type": "PenaltyDecision"},
  "then": {"property": "issuedBy"},
  "source": "deletion",
  "confidence": 0.90
}
```

Example (value-constraint):

```
{
  "id": "gov_val_amount_pos_003",
  "domain": "government",
  "type": "value_constraint",
  "if": {"entity_type": "PenaltyDecision", "property": "penaltyAmount"},
  "then": {"constraint": "> 0", "unit": "CNY"},
  "source": "augmentation",
  "confidence": 0.74
}
```

B. RuleTest Record Format

Each released case contains subject, subject type, relation, object, object type, a unique triple identifier, a data-quality field, and the expected detector outcome. The executable benchmark derives one of five defect-family labels using the same public detector definitions used to score predictions. The file contains no annotator identifiers or disagreement fields, so we make no inter-annotator agreement claim.

C. Artifact Index

The following repository artifacts correspond to the experiments reported in Section IV:

- `exps/paper2_dual_strategy_ablation/`: candidate-level deletion, augmentation, and dual-strategy ablation from stored LLM logs.
- `exps/paper2_rule_family_ablation/`: executable rule-family ablation on RuleTest-94.
- `exps/paper2_cooptimization/`: executable graph environment, DQN/Double-DQN runner, twenty checkpoints, transitions, learning histories, summaries, and figures.
- `exps/external_benchmark/`: deterministic TNEWS/CLUE KG repair and RuleTest-94 detector benchmark.
- `exps/api_llm_extraction_benchmark/`: API-backed TNEWS/CLUE extraction-path benchmark.
- `exps/shacl_baseline/`: SHACL-style structural validation baseline.
- `docs/paper2_experiment_reproducibility.md`: experiment-to-script mapping and boundary notes.

Candidate-level ablations, local repair benchmarks, and the RL benchmark can be rerun without API calls. The API-backed extraction benchmark depends on the configured model endpoint and may vary across model versions.

References

- [1] R. Das, S. Dhuliawala, M. Zaheer, L. Vilnis, I. Durugkar, A. Krishnamurthy, A. Smola, and A. McCallum, “Go for a walk and arrive at the answer: Reasoning over paths in knowledge bases using reinforcement learning,” in *International Conference on Learning Representations*, 2018.
- [2] W. Xiong, T. Hoang, and W. Y. Wang, “DeepPath: A reinforcement learning method for knowledge graph reasoning,” in *Proceedings of EMNLP*, 2017.
- [3] X. V. Lin, R. Socher, and C. Xiong, “Multi-hop knowledge graph reasoning with reward shaping,” in *Proceedings of EMNLP*, 2018.
- [4] W. Chen *et al.*, “Diva: Interpretable multi-hop reasoning via variational inference,” in *Proceedings of ACL*, 2021.
- [5] X. Chen, Y. Wang, and Z. Liu, “Reinforcement learning for knowledge graph construction,” in *Proceedings of AAAI*, 2020.
- [6] H. Zhu *et al.*, “Reinforcement learning for entity alignment,” in *Proceedings of IJCAI*, 2022.
- [7] W. Zhang *et al.*, “Adaptive knowledge graph construction with reinforcement learning,” in *Proceedings of CIKM*, 2021.
- [8] M. Ranzato *et al.*, “Sequence level training with recurrent neural networks,” in *International Conference on Learning Representations*, 2016.
- [9] R. Paulus, C. Xiong, and R. Socher, “A deep reinforced model for abstractive summarization,” in *International Conference on Learning Representations*, 2018.
- [10] J. Li *et al.*, “Deep reinforcement learning for dialogue generation,” in *Proceedings of EMNLP*, 2016.
- [11] J. Wei *et al.*, “Chain-of-thought prompting elicits reasoning in large language models,” *Advances in Neural Information Processing Systems*, 2022.
- [12] OpenAI, “Gpt-4 technical report,” *arXiv preprint arXiv:2303.08774*, 2023.
- [13] A. Q. Jiang *et al.*, “Mistral 7b,” *arXiv preprint arXiv:2310.06825*, 2023.
- [14] T. Detrmers *et al.*, “Qlora: Efficient finetuning of quantized llms,” in *Advances in Neural Information Processing Systems*, 2023.
- [15] P. Lewis *et al.*, “Retrieval-augmented generation for knowledge-intensive nlp tasks,” in *Advances in Neural Information Processing Systems*, 2020.
- [16] L. Yao *et al.*, “Kg-bert: Bert for knowledge graph completion,” in *arXiv preprint arXiv:1909.03193*, 2019.
- [17] X. Wang *et al.*, “Kepler: A unified model for knowledge embedding and pre-trained language representation,” in *Transactions of the Association for Computational Linguistics*, 2021.
- [18] S. Pan, L. Luo, Y. Wang, C. Chen, J. Wang, and X. Wu, “Unifying large language models and knowledge graphs: A roadmap,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 36, no. 7, pp. 3580–3599, 2024.
- [19] S. Yao *et al.*, “React: Synergizing reasoning and acting in language models,” in *International Conference on Learning Representations*, 2023.
- [20] —, “Tree of thoughts: Deliberate problem solving with large language models,” in *Advances in Neural Information Processing Systems*, 2023.
- [21] S. Auer *et al.*, “Dbpedia: A nucleus for a web of open data,” in *International Semantic Web Conference*, 2007.
- [22] F. M. Suchanek *et al.*, “Yago: A core of semantic knowledge,” in *Proceedings of the 16th International Conference on World Wide Web*, 2007.
- [23] H. Knublauch and D. Kontokostas, “Shapes constraint language (shacl),” *W3C Recommendation*, 2017.
- [24] E. Prud’hommeaux *et al.*, “Shape expressions: An rdf validation and transformation language,” *Proceedings of the 10th International Conference on Semantic Systems*, 2014.
- [25] L. A. Galárraga *et al.*, “Amie: Association rule mining under incomplete evidence in ontological knowledge bases,” in *Proceedings of the 22nd International Conference on World Wide Web*, 2013.
- [26] S. Ortona *et al.*, “Rudik: Rule discovery in knowledge bases,” in *Proceedings of the VLDB Endowment*, 2018.
- [27] F. Yang *et al.*, “Differentiable learning of logical rules for knowledge base reasoning,” in *Advances in Neural Information Processing Systems*, 2017.
- [28] Y. Lin *et al.*, “Llm-based knowledge graph quality assessment and enhancement,” *Proceedings of AAAI*, 2025.
- [29] H. Paulheim and C. Bizer, “Improving the quality of linked data using statistical distributions,” *International Journal on Semantic Web and Information Systems*, 2014.
- [30] A. Zaveri, A. Rula, A. Maurino, R. Pietrobon, J. Lehmann, and S. Auer, “Quality assessment methodologies for linked open data,” *Semantic Web*, vol. 7, no. 1, pp. 63–93, 2016.